Plano Estratégico de Evolução: Recon AI Automation (AS IS para TO BE)

1. Avaliação Estratégica: Começar do Zero vs. Evoluir o Projeto Atual (AS IS)

Decisão Recomendada: Evolução Gradual (Abordagem Híbrida / Refatoração Progressiva)

- **Motivação:** O script Shell atual (`recon-claude-grok.sh`) já oferece um pipeline funcional, maduro e altamente testado para automação de ferramentas CLI consagradas (Subfinder, Amass, Assetfinder, Dnsx, Httpx, Katana, Gau, Waybackurls, Nuclei). Começar do zero do ponto de vista operacional significaria descartar uma lógica de encadeamento e tratamento de erros já validada em ambiente de produção.
- **Estratégia de Transição:** O script Bash atual deve ser mantido como o "engine de execução legado" enquanto a nova arquitetura modular em Python é construída ao seu redor, migrando gradualmente os módulos Bash para funções/módulos Python assíncronos de forma transparente e controlada.

Comparativo de Abordagens

Critério	Reescrita Total (Greenfield)	Evolução Gradual (Abordagem Recomendada)
Tempo até o Primeiro Valor	Alto (semanas sem entregas funcionais)	Imediato (preserva operação do script atual)
Risco Operacional	Elevado (risco de regressão e bugs)	Baixo (migração modular e testada por fase)
Aproveitamento de Lógica	Nulo	Máximo (reutiliza encadeamentos validados)
Complexidade de Transição	Alta reorganização inicial	Gradual e adaptável ao longo do ciclo

2. Estratégias de Arquitetura, Interface e Infraestrutura

Estratégia 1: Interface de Usuário (CLI Avançada vs. Web GUI)

- **Recomendação:** Arquitetura Híbrida (CLI First com Web Dashboard Opcional).
- **CLI Avançada (Rich / Typer):** Manter o foco primário em CLI para permitir execução rápida, scripts remotos em VPS/VMs e integração via SSH. Utilizar bibliotecas como `Rich` e `Typer` para interfaces limpas, interativas e coloridas no terminal.
- **Web GUI (FastAPI + Streamlit / React):** Oferecer uma interface web leve para visualização de relatórios, gráficos de ativos, histórico de varreduras e triage de achados auxiliado por IA.

Estratégia 2: Conteinerização e Portabilidade (Docker & Docker Compose)

- **Recomendação:** Conteinerização Completa Multi-stage (Docker + Docker Compose).
- **Vantagens:** Resolve totalmente o gerenciamento de dependências de ferramentas Go/Python e evita conflitos de ambiente no Kali Linux, macOS ou ecossistemas de nuvem. Permite subir o ambiente completo (Orquestrador, Banco de Dados, Ollama/Local AI, Dashboard) com um único comando (`docker-compose up`).

Estratégia 3: Arquitetura do Sistema e Integração de Inteligência Artificial

- **Orquestrador Central em Python:** Substituir o encadeamento Bash por um orquestrador Python robusto com suporte a concorrência assíncrona (`asyncio` / `Celery`).
- **Persistência de Dados (SQLite / PostgreSQL):** Armazenar ativos e resultados em banco de dados relacional para viabilizar Recon Diferencial (identificar novos subdomínios ou alterações de estado entre execuções).
- **Camada Abstrata de IA (Provider Agnostic):** Criar um módulo de IA unificado capaz de alternar dinamicamente entre provedores gratuitos em nuvem (Gemini API, Groq) e modelos locais rodando no Apple Silicon ou GPUs dedicadas via Ollama (ex.: Mistral, Llama 3).

3. Planejamento de Transição (Roadmap do AS IS para o TO BE)

Cronograma Geral de Execução

Fase	Escopo Principal	Janela Estimada	Entregáveis Chave
Fase 1	Estabilização e Wrapper Python	Semanas 1 – 2	Dockerfile base, Wrapper CLI, Parsers JSON/SQLite
Fase 2	Camada de IA e Análise Semântica	Semanas 3 – 4	Módulo de IA unificado, Triage de Nuclei, Análise JS
Fase 3	Expansão de Capacidades de Recon	Semanas 5 – 6	Integração Naabu/Nmap, Gowitness e FFuF
Fase 4	Interface, Alertas e Recon Diferencial	Semanas 7 – 8	Engine de Recon Diferencial, Webhooks, Dashboard

Detalhamento das Fases

Fase 1: Estabilização e Wrapper Python do AS IS (Semanas 1 - 2)

- Empacotamento do script `recon-claude-grok.sh` atual dentro de uma estrutura de projeto Python bem definida.
- Criação de parser JSON/SQLite para capturar e estruturar a saída de cada fase do script Bash.
- Dockerização do ambiente base (instalação automatizada das ferramentas Go e dependências de ambiente).

Fase 2: Integração da Camada de IA e Análise Semântica (Semanas 3 - 4)

- Implementação do módulo Python de integração com APIs de IA (Gemini / Groq) e Ollama local.
- Integração de análise inteligente no código JavaScript (extração e validação semântica de tokens, chaves e endpoints).
- Módulo de Triage de Vulnerabilidades para análise automática dos resultados do Nuclei e eliminação de falsos positivos.

Fase 3: Expansão de Capacidades de Recon (Semanas 5 - 6)

- Adição do módulo de Port Scanning adaptativo (Naabu / Nmap).

- Adição do módulo de capturas visuais automatizadas (Gowitness).
- Implementação do módulo de Fuzzing de Conteúdo direcionado (FFuF).

Fase 4: Interface, Alertas e Recon Diferencial (Semanas 7 - 8)

- Sistema de notificações em tempo real via Webhooks configuráveis (Telegram / Discord).
- Banco de dados relacional e motor de Recon Diferencial (detecção e alerta de mudanças na superfície de ataque).
- Dashboard Web opcional para visualização gráfica e gestão centralizada de ativos.

Documento gerado como relatório técnico de apoio ao planejamento estratégico do projeto Recon AI Automation.